FinIntell AI — Full-Stack Freelance Roadmap (v5)

Target Technology Stack: Java 17 LTS (Spring Boot 3) + React 19 (JavaScript / JSX)

> **Official GitHub Repository:** https://github.com/SHUBHAM-MAKODE/FinIntell-AI/tree/main
> *All source code, configuration files, Flyway SQL migrations, and documentation are tracked at this repository.*

1. Technology Stack Architecture (Java 17 & React JS)

Layer / Domain	Selected Technology	Version / Spec	Core Function & Role in SaaS
Backend Runtime	Java LTS	Java 17	Enterprise OOP, sealed classes, Records, text blocks
Backend Framework	Spring Boot	3.x	REST APIs, Jackson JSON serialization, HikariCP
Data Persistence	Spring Data JPA / Hibernate	Spring Boot 3	ORM data mapping, SQL queries, transactional integrity
Database Migration	Flyway Core	Standard	Version-controlled SQL schema creation (V1__init.sql)
Security & Auth	Spring Security	6.x	Stateless JWT token authentication, BCrypt, CORS filters
Validation & Utility	Jakarta Validation & Lombok	Standard	@NotNull/@Valid annotations, getter/setter code gen
AI Orchestration	Spring AI	OpenAI Starter	GPT-4o Vision OCR parsing, BeanOutputConverter, RAG
Frontend Library	React	19.x (JavaScript/JSX)	Single Page Application (SPA), hooks, context state
Build Tool	Vite	react template	Instant HMR development server & production JS bundler
HTTP Client	Axios	Latest	REST API integration & JWT Bearer token request interceptors
UI & Icons	Tailwind CSS & Lucide Icons	Latest	Utility-first dark slate styling & vector UI icons
Data Visualization	Recharts	Latest	Interactive spending trend area charts & category donut charts
Database Engine	PostgreSQL + pgvector	16.x	ACID financial relational storage + 1536-dim vector search
Containerization	Docker & Compose	Multi-stage	Self-contained application deployment containers

2. Sequential 20-Task Step-by-Step Execution Checklist

Phase 1: Backend & Database Foundation (Tasks 1–7)

• Task 1: Verify local Java 17, Node.js 20+, and PostgreSQL 16 installations; run 'CREATE DATABASE finintell_db;'.

• Task 2: Initialize Spring Boot 3 app ('finintell-backend') via Spring Initializr with Java 17, Web, JPA, Security, Validation, Lombok, Flyway.

• Task 3: Configure 'application.yml' for HikariCP database connections, JPA PostgreSQL dialect, and Flyway migration parameters.

• Task 4: Write Flyway migration 'V1__init_schema.sql' for 'users', 'categories', 'transactions', 'receipts', and 'vector_embeddings' (with pgvector).

• Task 5: Create Java 17 JPA Entities ('UserEntity', 'TransactionEntity') and Java Records for Request/Response DTOs.

• Task 6: Build Spring Data JPA repositories and Service classes handling business logic, user checks, and validation.

• Task 7: Build RestControllers ('/api/v1/transactions') with '@Valid' checks and '@ControllerAdvice' exception handler.

Phase 2: Security & React JavaScript Client (Tasks 8–12)

• Task 8: Implement Spring Security 6 filter chain, HMAC-SHA256 'JwtTokenProvider', BCrypt password encoder, and auth endpoints.

• Task 9: Initialize React 19 JavaScript app via Vite ('npm create vite@latest finintell-frontend -- --template react') and install Axios, React Router, Tailwind, Recharts, Lucide.

• Task 10: Configure Tailwind CSS slate theme, '.env' API base URLs, and 'axiosClient.js' with Bearer token request interceptors.

• Task 11: Create React 'AuthContext.jsx' state provider, login/signup forms, and protected route guards.

• Task 12: Build 'DashboardPage.jsx' featuring monthly KPI summary cards, Recharts spending trend charts, and expense tables.

Phase 3: AI Engine, Receipt OCR & Vector Search (Tasks 13–17)

• Task 13: Add Spring AI OpenAI starter dependency to 'pom.xml' and configure OpenAI API key environment variables.

• Task 14: Build receipt file upload controller and 'ReceiptParsingEngine' using Spring AI ChatClient + GPT-4o Vision OCR.

• Task 15: Refactor OCR receipt extraction using '@Async' background tasks and state tracking ('PENDING', 'EXTRACTED', 'FAILED').

• Task 16: Implement 'FinancialEmbeddingService' to generate 1536-dimensional vector embeddings for transaction history.

• Task 17: Build 'AIChatDrawer.jsx' sliding drawer component in React to query financial spending in natural language via RAG.

Phase 4: Testing, Docker & GitHub Showcase (Tasks 18–20)

• Task 18: Write JUnit 5 / Mockito unit tests for service classes and integration tests for REST API endpoints.

• Task 19: Construct multi-stage 'Dockerfile' for Spring Boot (JDK 17) and React (Nginx), orchestrated via 'docker-compose.yml'.

• Task 20: Push complete codebase to GitHub repository (https://github.com/SHUBHAM-MAKODE/FinIntell-AI/tree/main) and deploy to cloud (Render/Vercel).